- Create a **draw.io XML file you can import directly**
- Or generate a **clean professional diagram image** 👍

i have a dataset, how do i integrate them on the website

Great question—this is the **most important practical step** of your project. I'll explain it in **simple steps** so you can actually build it.

🌱 How to Integrate Your Dataset into Your Website

You basically need to move your data like this:

👉 **Dataset → Backend → Database → Website (Dashboard)**

🧩 Step-by-Step Guide

✅ Step 1: Prepare Your Dataset

- Make sure your dataset is in:
  - `.csv` (best option)
- Example columns:

```
timestamp, temperature, humidity, soil_moisture, light
```

✅ Step 2: Store Data in Database (MySQL)

1. Create a table:

```sql
CREATE TABLE readings (
    id INT AUTO_INCREMENT PRIMARY KEY,
    timestamp DATETIME,
    temperature FLOAT,
    humidity FLOAT,
    soil_moisture FLOAT,
    light FLOAT
);
```

2. Import your dataset:

- Use MySQL Workbench → "Import CSV"
- OR use Python (recommended)

✅ Step 3: Load Dataset Using Python (Easy Way)

Use Python + Pandas:

```python
import pandas as pd
import mysql.connector

# Load dataset
df = pd.read_csv("data.csv")

# Connect to MySQL
```

```
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="yourpassword",
    database="greenhouse"
)

cursor = conn.cursor()

# Insert data
for _, row in df.iterrows():
    cursor.execute("""
        INSERT INTO readings (timestamp, temperature, humidity, soil_moisture,
        VALUES (%s, %s, %s, %s, %s)
```

✅ Step 4: Create Backend API (C#)

Using ASP.NET Core:

Example API:

```
[HttpGet]
public IActionResult GetReadings()
{
    var data = _context.Readings.ToList();
    return Ok(data);
}
```

👉 This lets your website **fetch data from the database**

✅ Step 5: Connect Website (Frontend)

Using JavaScript:

```
fetch("https://localhost:5001/api/readings")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

✅ Step 6: Show Data as Graphs

Using Chart.js:

```
new Chart(ctx, {
  type: 'line',
  data: {
    labels: data.map(d => d.timestamp),
    datasets: [{
      label: 'Temperature',
      data: data.map(d => d.temperature)
    }]
  }
});
```

🔄 Simple Flow

```
CSV Dataset
   ↓
Python → MySQL
   ↓
C# API
   ↓
JavaScript Website
   ↓
Chart.js Graphs
```

⚡ Easier Option (If You're Short on Time)

Skip backend and use CSV directly:

```
fetch("data.csv")
  .then(response => response.text())
  .then(csv => {
    // parse and display
  });
```

👉 BUT:

- ❌ Not good for large projects
- ❌ No database
- ✔️ OK for demo

🎯 Simple Summary